Vanishing Gradient Problem: An Activation Function that Seeks to Improve VGP

Yiling Zou

University of Rochester,Rochester NY,14627,USA.

Abstract. The performance of deep learning networks can easily be affected by the phenomenon of vanishing gradients. With this in mind, this paper mainly discusses an activation function different from ReLU and other functions that improve the vanishing gradient problem (VGP). Influenced by parameterized functions and equations, we can set an activation function that allows for manual parameter changes. In this way, we can avoid the vanishing gradient situation when the input received by backpropagation is close to 0. With such an activation function, we can manually avoid vanishing gradients; in the experimental part, this paper uses the MNIST dataset to verify the effectiveness of the proposed method. The experiments demonstrate that the proposed method can alleviate the vanishing gradient phenomenon to a certain extent.

Keywords: VGP, activation function, back propagation, ReLU, parameterized.

1. Introduction

1.1 Development of AI

Not long ago, OpenAI launched GPT-4, which has greater functionality then GPT-3 [1] in most parts. Many people are astonished by the way a chatbot can talk and all the questions it can answer. Of course, this chatbot is not a person. But how can it interpret and answer questions in a mostly correct and academic way? Of course, this involves neural networking, Large Language Models, deep learning, natural language processing and other technologies.

Artificial neural network(or neural network) [2] is by far one of the most popular machine learning models that are inspired by biological neural networks.

If we go deeper into neural networks, we can see that there are different types of neural networks, including CNN [3], most commonly used to analyze visual imagery; RNN [4] , mostly used to music composition, robot control, and human action recognition; Autoencoders, used to process generative data models, and so on. Each type of neural network focus on their specific tasks. However, no matter what type of neural network it is, all the NNs suffer from vanishing gradient problems [5]. The vanishing gradients problem, a challenge in training deep neural networks, affects learning speed, model performance, and the ability to train deep architectures. Therefore, how to alleviate the vanishing gradient phenomenon is one of the mainstream issues currently studied in the academic community.

1.2 Vanishing Gradients Problem

The vanishing gradients problem is a difficulty encountered in training deep neural networks using gradient-based optimization methods. These methods adjust the parameters (weights) of a model based on the gradient of the loss function with respect to the parameters. Essentially, these methods use the gradient (the first derivative of the loss function) to decide how much to adjust each parameter during each iteration of training.

The vanishing gradients problem occurs because the gradients often become very small as they are backpropagated through the layers of a deep network. This is due to the multiplication of many small numbers (below one), which results in a very small gradient.

The consequence of this is that the parameters in the earlier layers of the network learn very slowly compared to the later layers (those closer to the output). This slow learning makes the training process inefficient and can even lead to the model failing to converge to a good solution.

1.3 Framework

This essay would suggest a function that could improve the vanishing gradient problem. Part Two would focus on the restatement of Vanishing Gradient Problems. Part Three would give common existing activation functions that improves the VGP. Part Four would be an explanation and statement of my activation function to the VGP. Part Five would be a comparison between my activation function and other activation funtions that solve VGP.

2. Impacts of the Vanishing Gradients Problem

The vanishing gradient problem poses a considerable challenge to the training process and performance of deep neural networks, manifesting in several key ways:

2.1 Slowed Training

The vanishing gradient issue often leads to a notable decrease in the learning speed of the initial layers in deep neural networks, compared to the later ones. As gradients are backpropagated through the layers, they tend to diminish, resulting in minuscule updates to the weights in the earlier layers. Consequently, the learning rate for these layers decreases significantly, slowing down the overall training process.

2.2 Suboptimal Convergence

Another repercussion of the vanishing gradient problem is the potential for the model to fail to converge optimally. When weight updates become increasingly small, the weights may cease changing significantly before achieving an ideal set of values. This could prevent the model from identifying underlying patterns in the data, leading to subpar performance.

2.3 Challenges in Learning Long-Range Dependencies

In relation to recurrent neural networks (RNNs), the vanishing gradient problem hampers the network's ability to learn long-range dependencies within data sequences. Information contribution decays geometrically over time due to the multiplication of small gradients. This limitation can significantly impede the performance of RNNs in tasks like language modeling, where understanding context from distant past inputs is crucial.

2.4 Uneven Parameter Updates

The diminished magnitude of gradients for the earlier layers leads to more aggressive updates in the parameters of the network's later layers. This imbalance during training often results in a less than ideal overall configuration of the model's parameters, potentially reducing the model's effectiveness.

The vanishing gradient problem remains a significant hurdle in training deep neural networks. However, numerous strategies have been proposed to alleviate this issue, including the utilization of ReLU activation functions, suitable weight initialization techniques, batch normalization, and advanced network architectures like ResNets.

3. Problem Restatement

The problem of the vanishing gradient was first discovered by Sepp (Joseph) Hochreiter back in 1991 [5].When there are more layers in the network, the value of the product of derivative decreases until at some point the partial derivative of the loss function approaches a value close to zero, and the partial derivative vanishes.

To be specific, we define this problem in a more mathematical way. That is, we set up a loss function, call it L, giving the difference between the predicted value and the actual value. Here for simplicity and accuracy, we could assume that we apply softmax with cross entropy layer as the loss function (actually does not matter which loss function we pick). In addition, we choose the sigmoid

function as our activation function. What we are trying to do is to train this neural network to get the most accurate result. Thus, mathematically speaking, we need to find a local (global, if can be achieved) L with some proper gradient. However, during back propogation, the weight matrix W decays with the step of its gradient multiplied by a specified learning rate. Consequently, the vanishing gradient problem states the difficulty on training due to small gradient.

During back propogation, we have the following equations. Let W, X, B, f be their usual definition in neural networks. Let $Y = f(X)$ and $y_i = f(x_i)$.

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial w_i} \tag{1}$$

$$\frac{\partial L}{\partial y_{i-1}} = \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial x_i} \tag{2}$$

The two equations above illustrate the main theme of calculating the gradients. In back propagation, we update the parameter wi by the following equation.

$$W_{new} = W_{old} - \eta \cdot \frac{\partial L}{\partial w_{old}} \tag{3}$$

While η is the learning rate of this neural network. We can observe from the given equation that to keep W to be updated effectively and efficiently. We must make sure that $\partial L / \partial w_{old}$ is not small. Thus, we know that $\partial L / \partial y_i$ and $\partial y_i / \partial w_{old}$ should be kept large. Otherwise, Wnew would be nearly equal to Wold at some time, implying that the neural network is barely or not learning.

To be more intuitive, we could consider that the chain rule of derivatives. When the neural networks multiply all the partial derivatives together, the more value between 0 and 1, the more less we get the gradient. Finally, the network would reach a state which is not learning.

4 Activation Functions that Solve VGP

In 2006, Hinton et al. [6] describe an activation function that could improve this problem, which is rectified linear unit, namely ReLU. The following is ReLU:

$$f(x) = \max(0, x) \tag{4}$$

While it is true that ReLU [7] is convenient and improves the vanishing gradient problem by choosing 0 for negative values and constant 1 for positive values, The "dying ReLU" problem occurs when a neuron always outputs zero for all inputs during training. Once a ReLU neuron gets into this state, it is unlikely to recover because the gradient of the ReLU function is zero for negative inputs. Neurons can become "dead" if they consistently receive negative gradients during training, causing the weights to stagnate. Similarly, leaky ReLU [8] improves the problem of dead neurons, but it then comes back to the problem that the gradient is not uniform for negative values. Thus, the learning behavior would suffer from vanishing gradient when the negative slope is very small.

4. Parameterized Activation Function

Inspired by the leaky ReLU, we could choose more parameters to avoid the situation that one parameter dominates the gradient. We have

$$f(x) = -ae^{-bx^2} + 1 \tag{5}$$

We could always map the gradient to the range (0,1). While maintaining the mapping range, we could vary the gradient by a,b, which can speed up the learning process if we choose the right parameters a, b. To see the properties more clearly, we could check the gradient by calculating

$$\frac{\partial f}{\partial x} = -ae^{-bx^2} \cdot (-2bx) \tag{6}$$

This step could be easily derived from the chain rule of derivatives. Simplifying, we get

$$\frac{\partial f}{\partial x} = 2abe^{-bx^2}x \tag{7}$$

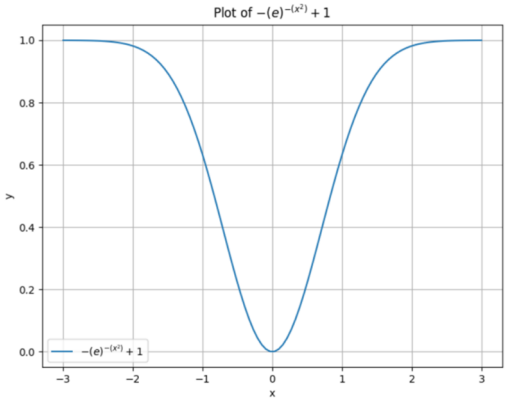

Figure 1 Parameterized Activation Function with a=b=1.

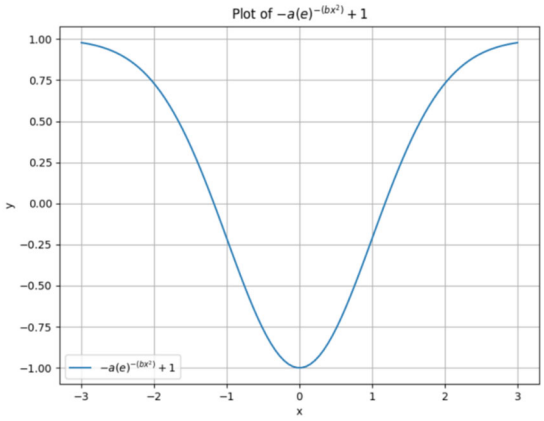

Figure 2 Parameterized Activation Function with a=2,b=1.

From Fig 1, Fig 2 and the calculation of the gradient, it is clear that this gradient can be changed by having various choices of a, b. Thus, we could introduce this Theorem: f(x) can vary the gradient with different a, b.

5. Experiments with MNIST

5.1 Introduction of MNIST

The MNIST dataset [9] is a large database of handwritten digits that is commonly used for training various image processing systems, particularly in the field of machine learning. It contains 60,000 training images and 10,000 testing images, each of which is a 28x28 grayscale image of a digit between 0 and 9. Its simplicity and size make it a popular starting point for deep learning tasks.

5.2 Parameters Setting

Here we choose to use Convolutional Neural Network as the testing Neural Network implemented in Python. There are 2 hidden layers in the test code and output of the last hidden layer is then sent to the softmax layer before calculating the cross-entropy loss. No regularization techniques are included in this process. Parameters a and b would be set to some values. Later we would use P(a,b) to represent parameterized activation function with parameter a,b.

5.3 Results and analysis

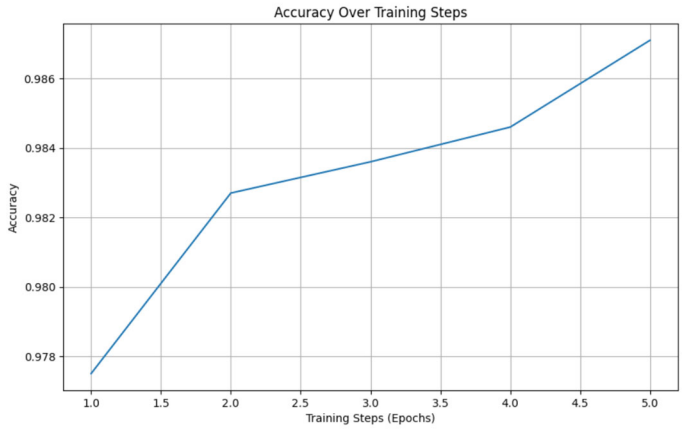

Figure 3 The Classification Performance with ReLU.

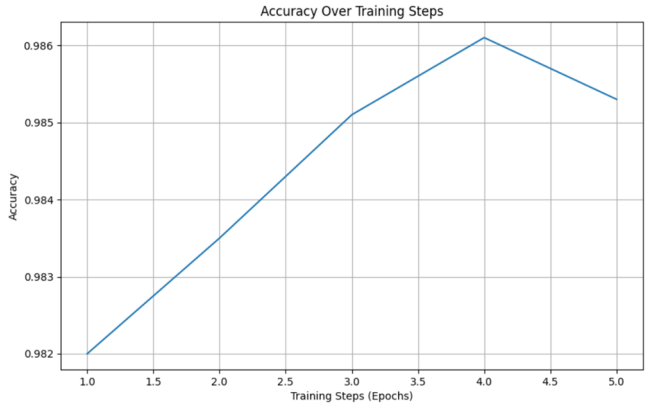

Figure 4 Parameterized Activation Function with a=1,b=5.

Fig 3 and Fig 4 demonstrate a comparison when choosing ReLU and the parameterized activation function as the activation function. We can tell that the performance of ReLU and P(a,b) cannot be determined obviously. We need to test more pairs of (a,b) to get the generalized result.

Table 1 Accuracy of Different Activation Function Parameters.

Parameterized Activation Functions	Accuracy
ReLU	98.39%
P(1,0.5)	98.45%
P(1,1)	98.37%
P(1,5)	98.53%
P(0.9,5)	95.49%
P(1.1,5)	98.71%

We've collected the parameterized activation functions, ReLU and their corresponding accuracy. We could readily observe that P(1,0.5),P(1,5),P(1.1,5) performs better than ReLU while P(1,1), P(0.9,5) performs poorer than ReLU. Notice that P(0.9,5) is the worst, implying that we may not choose a low a for this activation function, but more tests need to be completed to prove this hypothesis.

From the above table, we can see that a good choose of (a,b) is important. Otherwise, we may get a even lower accuracy compared to that of ReLU.

6. Conclusion and Future Work

We propose a parameterized activation function P(a,b) that tries to solve the VGP. Through the experiment conducted, a good choice of value pair (a,b) do perform better than common activation functions such as ReLU. However, as long as this neural network is based on graident-learning. The vanishing gradient problem would continue to exist. This function proposed in this essay only delays the VGP but did not solve the problem completely. To check generality of this function is promising, which requires more tests on different datasets using different networks such as RNN.

The vanishing gradient problem has always been an important challenge in the field of deep learning, but the future prospects are promising. Initial coping strategies, such as the use of ReLU activation functions, improved initialization techniques, and the proposal of residual network (ResNets) architecture, have alleviated this problem to a certain extent. In the future, we expect more research to focus on the nature of the vanishing gradient problem, including a more in-depth theoretical framework, and possibly new optimization algorithms. In addition, advances in self-supervised learning and unsupervised learning may provide new methods to deal with gradient problems in deep neural networks, and more advanced network architecture designs may also emerge, which may allow us to have a deeper understanding of the vanishing gradient problem.

References

[1] T. Brown, B. Mann, N. Ryder, et al., “Language models are few-shot learners,” Advances in neural information processing systems 33, 1877–1901 (2020).

[2] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” nature 521(7553), 436–444 (2015).

[3] J. Bouvrie, “Notes on convolutional neural networks,” (2006).

[4] L. R. Medsker and L. Jain, “Recurrent neural networks,” Design and Applications 5(64-67), 2(2001).

[5] S. Hochreiter, “The vanishing gradient problem during learning recurrent neural nets and problem solutions, International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems 6(02), 107–116 (1998).

[6] G. E. Hinton, S. Osindero, and Y.-W. Teh, “A fast learning algorithm for deep belief nets,” Neural computation 18(7), 1527–1554 (2006).

[7] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in Proceedings of the 27th international conference on machine learning (ICML-10), 807–814 (2010).

[8] A. L. Maas, A. Y. Hannun, A. Y. Ng, et al., “Rectifier nonlinearities improve neural network acoustic models,” in Proc. icml, 30(1), 3, Atlanta, GA (2013).

[9] L. Deng, “The mnist database of handwritten digit images for machine learning research [best of the web],” IEEE signal processing magazine 29(6), 141–142 (2012).